

Microservices

Building Microservice Systems

Sharif University of Technology, Spring 2026

Ali Abrishami

Seyyed Hossein Mirhosseini



What Are Microservices?



Why microservices?

- An approach to distributed systems that promote the use of **finely grained services** that can be changed, deployed, and released independently.
- For organizations that are moving toward more **loosely coupled** systems, with **autonomous teams** delivering user-facing functionality.
- Gives a lot of **flexibility** to ensure the system can change to meet the needs of the users.



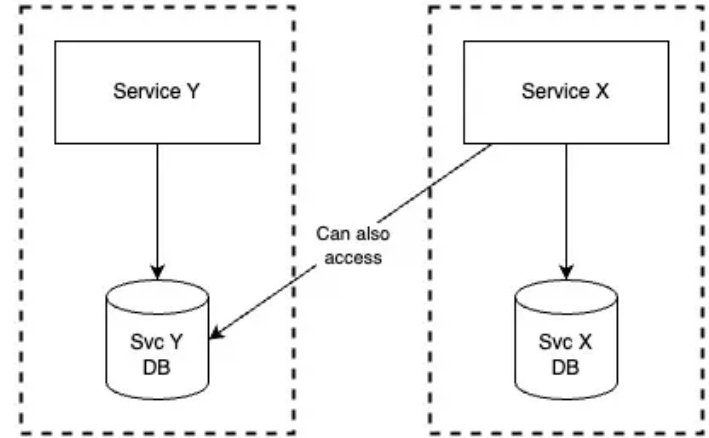
Microservices Definition

- *Microservices* are **independently** releasable services that are modeled around a **business domain**.
- A service encapsulates functionality and makes it accessible to other services via networks.
- You construct a more complex system from these building blocks.
- **Example:** One microservice might represent *inventory*, another *order management*, and yet another *shipping*.
But together they might constitute an entire *ecommerce system*.

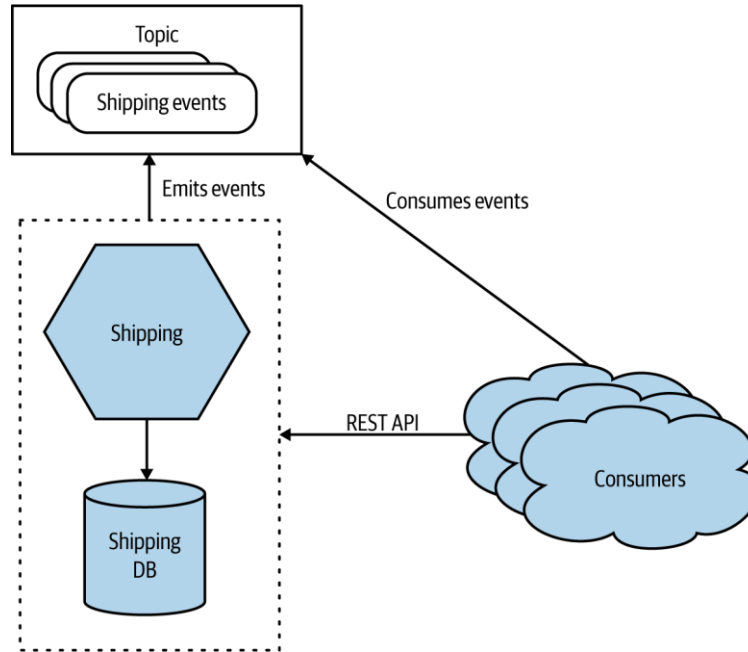


A Single Microservice

- A single microservice is treated as a **black box**.
- It hosts business functionality on one or more network endpoints.
 - For example, a queue or a REST API.
- Internal implementation details are entirely hidden from the outside world.
- So, the use of shared databases is **avoided** in most circumstances.



A Single Microservice (cont.)



A microservice exposing its functionality over a REST API and a topic



Information Hiding

- Microservices embrace the concept of information hiding.
- **Information hiding** means hiding as much information as possible inside a component and exposing **as little as possible** via external interfaces.
- Changes inside a microservice boundary shouldn't affect an upstream consumer.
 - For enabling independent releasability of functionality.



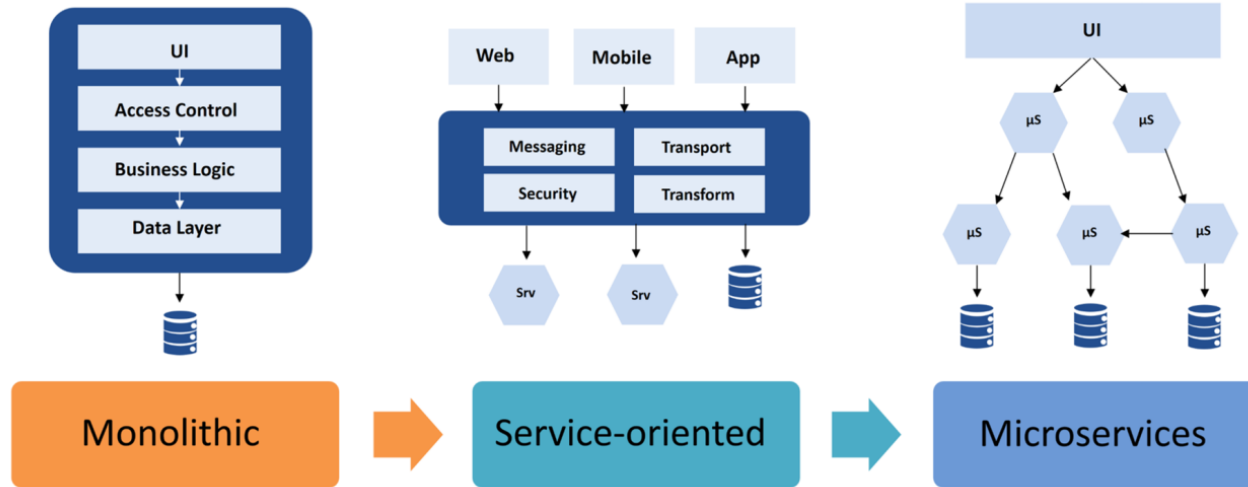
SOA vs. Microservices

- *Service-oriented architecture* (SOA) is a design approach in which multiple services collaborate to provide a certain set of capabilities.
- SOA emerged as an approach to combat the challenges of large, monolithic applications. Theoretically we can **replace** one service with another without anyone knowing.
- The microservice approach has emerged from real-world use, taking our better understanding of systems and architecture to do SOA well.
- Think of microservices as being **a specific approach** for SOA.
 - In the same way that Extreme Programming (XP) or Scrum is a specific approach for Agile software development.



SOA vs. Microservices (cont.)

Evolution of Software Architectures



Key Concepts of Microservices

1. Independent Deployability
2. Modeled Around a Business Domain
3. Owning Their Own State
4. Size
5. Flexibility
6. Alignment of Architecture and Organization



1) Independent Deployability

- Make a change to a microservice, deploy it, and release that change to our users, without having to deploy any other microservices.
- This is the **most important** concept in a micro service system.
- To ensure independent deployability, we need to make sure our microservices are **loosely coupled**.
- This means we need explicit, well-defined, and stable **contracts** between services.
- To achieve independent deployability, there are so many other things you have to get right that in turn have their own benefits.



2) Modeled Around a Business Domain

- Techniques like ***domain-driven design*** can allow you to structure your code to better represent the real-world domain that the software operates in.
- By modeling services around business domains, we can make it easier to roll out new functionality and to recombine microservices in different ways to deliver new functionality.
- Prioritize high cohesion of **business functionality** over high cohesion of **technical** functionality.



3) Owning Their Own State

- Microservices should **avoid** the use of shared databases.
- If a microservice wants to access data held by another microservice, it should go and **ask** that second microservice for the data.
- This gives the microservices the ability to decide what is shared and what is hidden.
- Hiding internal state in a microservice is analogous to the practice of **encapsulation** in object-oriented (OO) programming.
- Sharing databases is **one of the worst** things you can do if you're trying to achieve independent deployability.



4) Size

- *A microservice should be as big as my head.* (James Lewis)
- A microservice should be kept to the size at which it can be **easily understood**.
- When you are first starting out, it's much more important that you focus on two key things:
 - First, how many microservices can you handle? (As you have more services, the complexity of your system will increase.)
 - Second, how to define microservice boundaries.
- Note that the concept of size is actually one of the **least** interesting aspects.



5) Flexibility

- ***Microservices buy you options.*** (James Lewis)
- In software engineering, ***keeping options open*** means designing systems so that future changes are easier and less risky.
- They have a cost, and you must decide whether the cost is worth the options you want to take up:
 - Organizational
 - Technical
 - Scale
 - Robustness

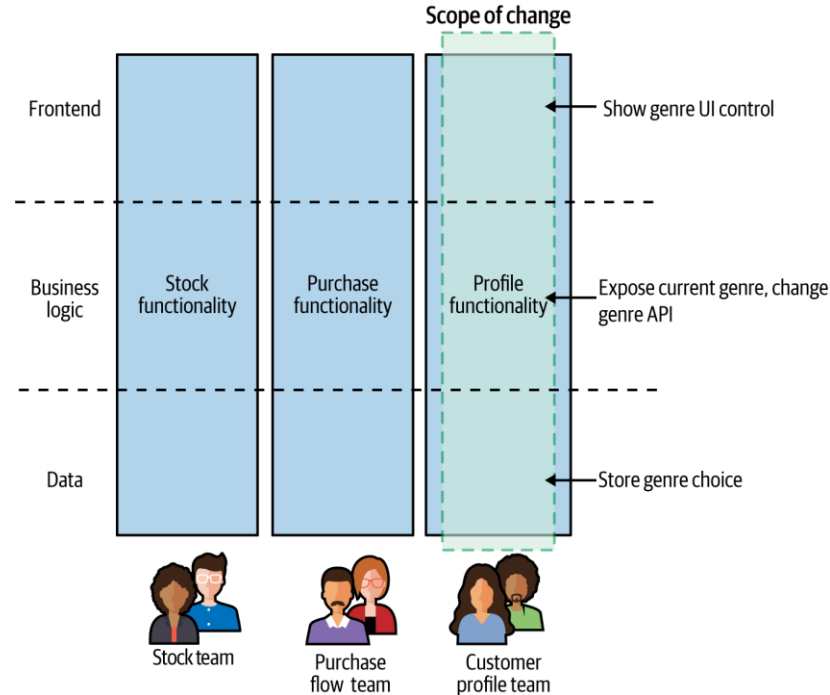


6) Alignment of Architecture and Organization

- *Organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations.*
(Melvin Conway)
- Most changes that we are asked to make to our system relate to changes in **business functionality**.
- Rather than a horizontally layered architecture and organization, we instead break down our organization and architecture along vertical business lines.



Alignment of Architecture and Organization



stream-aligned teams



Alignment of Architecture and Organization (cont.)

- This concept of a team owning an end-to-end slice of **user-facing functionality** is gaining traction.
- A stream-aligned team is a team aligned to a single, **valuable stream of work**... The team is empowered to build and deliver customer or user value as quickly, safely, and independently as possible, without requiring **hand-offs** to other teams to perform parts of the work.
(*Team Topologies book*)



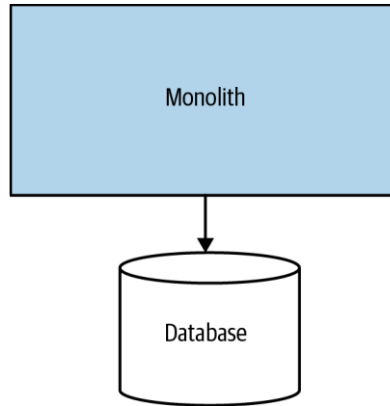
The Monolith

- When all functionality in a system must be deployed together, it is considered a monolith.
- Three types of monoliths:
 1. The Single-Process Monolith
 2. The Modular Monolith
 3. The Distributed Monolith



The Single-Process Monolith

- A system in which all of the code is deployed as **a *single process***.
- You may have multiple instances of this process for robustness or scaling reasons, but fundamentally all the code is packed into a **single process**.



A single-process monolith



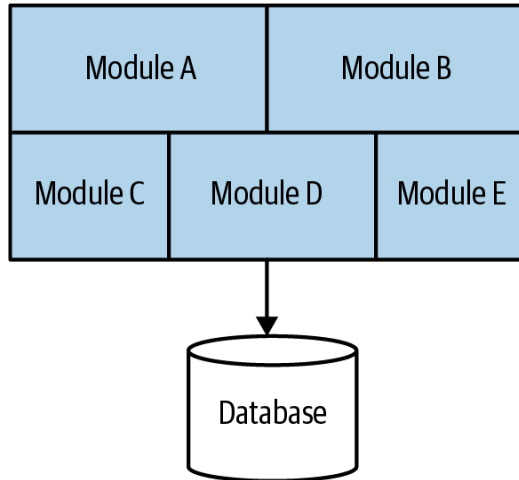
The Modular Monolith

- The *modular monolith* is a variation in which the single process consists of **separate modules**.
- Each module can be worked on independently, but all still need to be **combined** together for deployment.
- If the module boundaries are well defined, it can allow for a high degree of parallel work, while avoiding the challenges of the more distributed microservice architecture by having a much simpler deployment topology.

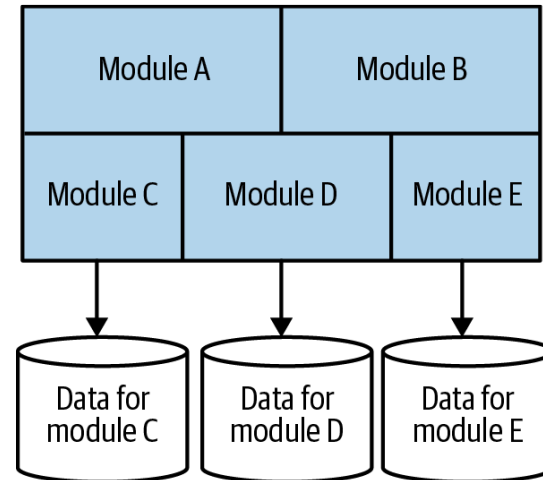


The Modular Monolith (cont.)

- For many organizations, the modular monolith can be an excellent choice.



modular monolith



A modular monolith with a decomposed database



The Distributed Monolith

- A *distributed monolith* is a system that consists of multiple services, but for whatever reason, the entire system must be **deployed together**.
- A distributed monolith has **all** the disadvantages of a distributed system, *and* the disadvantages of a single-process monolith.



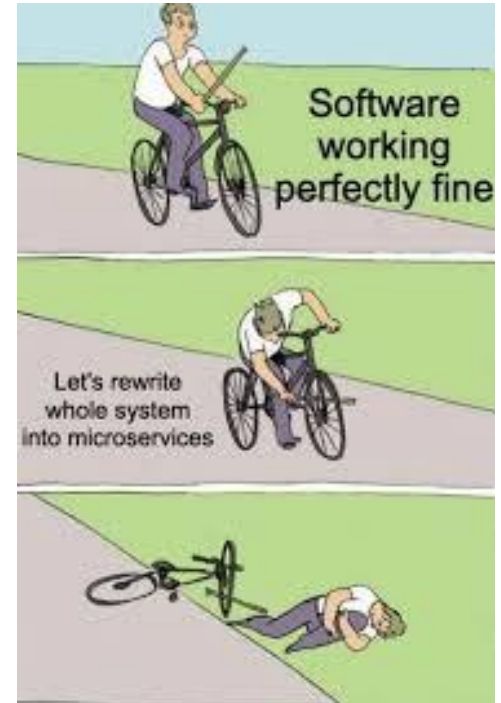
Monoliths and Delivery Contention

- As more and more people work in the same place, they get in one another's way. For example:
 - Different developers wanting to change the same piece of code
 - Different teams wanting to push functionality live at different times
- A microservice architecture does give you more concrete boundaries, giving you much more flexibility.



Advantages of Monoliths

- Much **simpler deployment** topology.
- Much simpler developer workflows, and monitoring, troubleshooting.
- Activities like end-to-end testing can be greatly simplified as well.
- Monoliths can also simplify **code reuse** within the monolith itself.
- So, it is the sensible **default choice** as an architectural style.



Enabling Technology

- Technology has played a large part in the adoption of microservices as a concept.
- Some important technologies for supporting microservice architecture:
 1. Log Aggregation and Distributed Tracing
 2. Containers and Kubernetes
 3. Streaming
 4. Public Cloud and Serverless

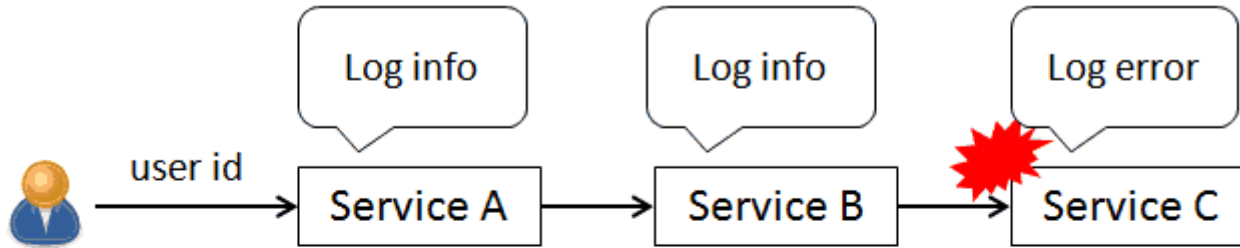


1) Log Aggregation and Distributed Tracing

- At first, Understand how your system is behaving in a production setting.
- A log aggregation system is a **prerequisite** for adopting a microservice architecture.
- Log aggregation tools can be made even more useful by implementing **correlation IDs**, in which a single ID is used for a related **set of service calls**.
 - For example, the chain of calls that might be triggered due to user interaction.



1) Log Aggregation and Distributed Tracing (cont.)



[Service A][Info] received request from [user id]...

...

[Service B][Info] received request to ...

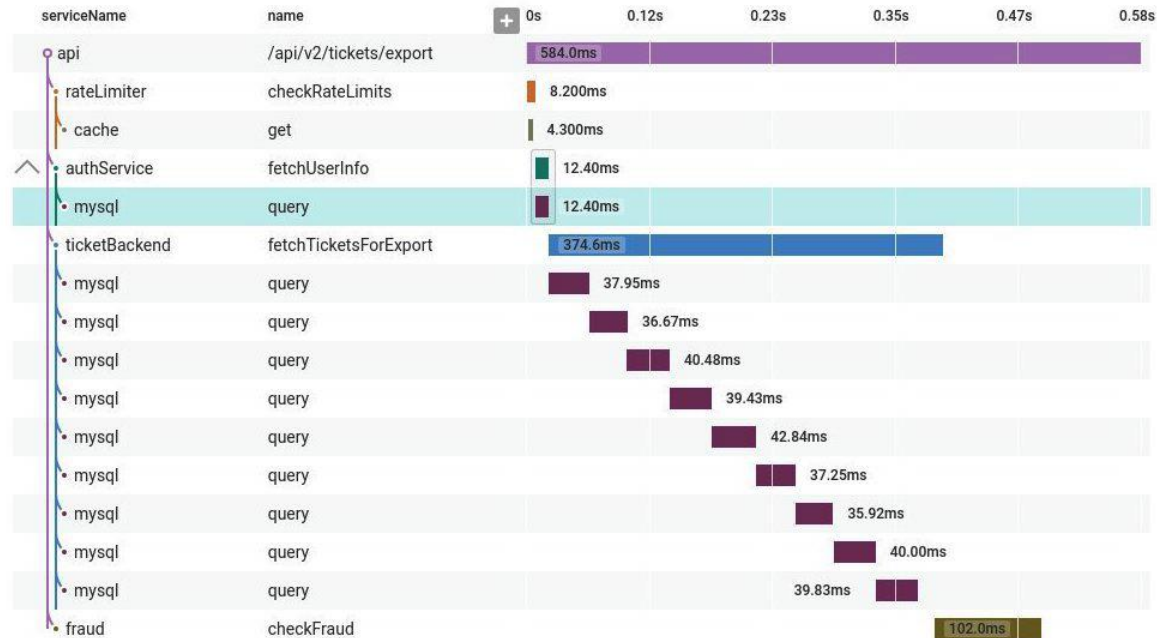
...

[Service C][Error] failed to process request...



1) Log Aggregation and Distributed Tracing (cont.)

Query at 5/31 3:35PM > Trace e6ee35b206e1c9e5



A distributed trace shown in Honeycomb

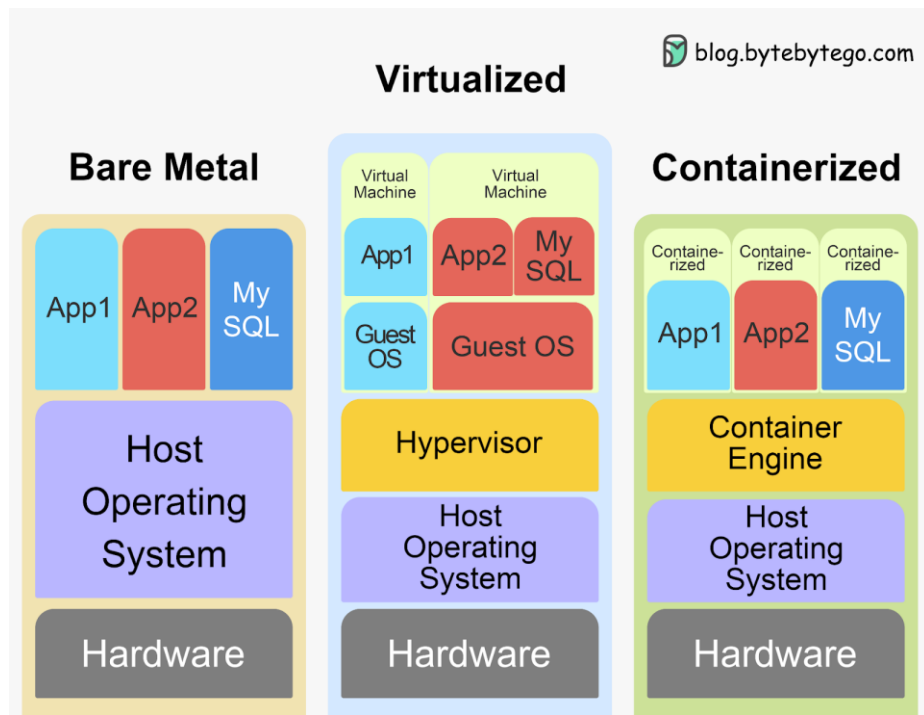


2) Containers and Kubernetes

- **Virtualization** is one way to create isolated execution environments on existing hardware.
- But normal virtualization techniques can be **quite heavy** when we consider the size of our microservices.
- **Containers**, on the other hand, provide a much more **lightweight** way to provision isolated execution for service instances.
- This results in faster spin-up times for new container instances, along with being much more cost effective for many architectures.
- **Container orchestration** platforms like Kubernetes allow you to manage these containers across lots of underlying machines.



2) Containers and Kubernetes (cont.)

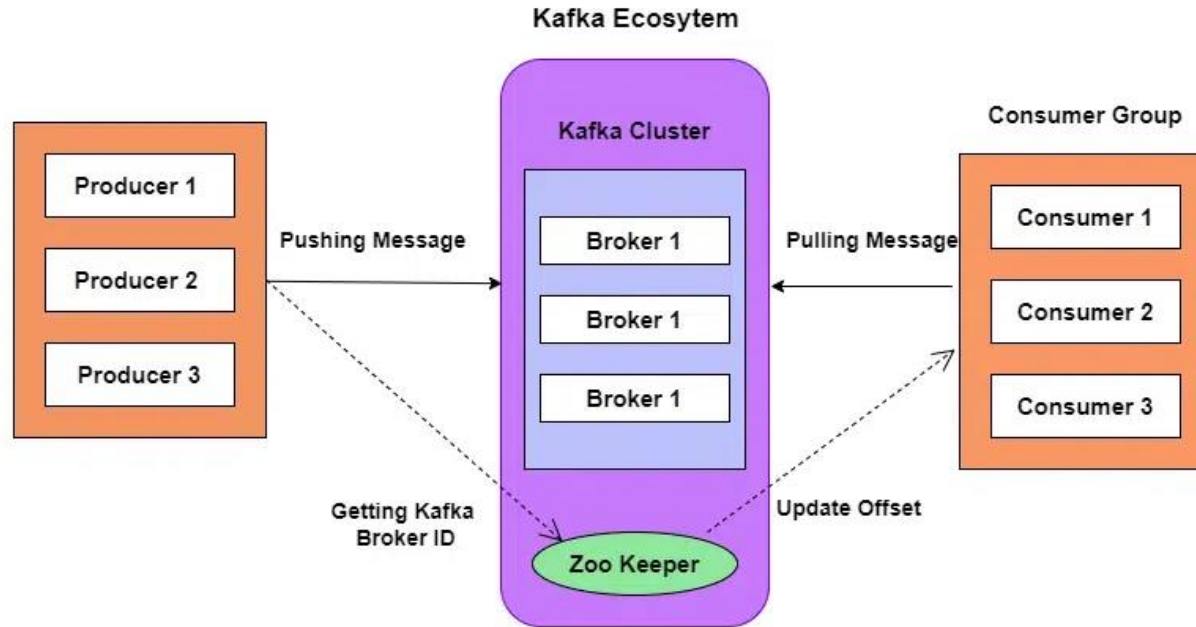


3) Streaming

- We still need to find ways to **share data** between microservices.
- Products that allow for the easy streaming and processing of what can often be **large volumes of data** have therefore become popular.
- For many people, ***Apache Kafka*** has become the de facto choice for streaming data in a microservice environment, and for good reason.
- Capabilities such as **message permanence**, **compaction**, and the ability to **scale** to handle large volumes of messages can be incredibly useful.



3) Streaming (cont.)



4) Public Cloud and Serverless

- Public cloud providers offer a huge array of managed services and deployment options for managing your application.
- The main three providers are:
 - Google Cloud,
 - Microsoft Azure,
 - And Amazon Web Services (AWS).
- The products that sit under the banner of **serverless**.
- These products hide the underlying machines, allowing you to work at a **higher level of abstraction**.



Advantages of Microservices

- By combining the concepts of information hiding and domain-driven design with the power of distributed systems, microservices can help deliver significant gains over other forms of distributed architectures:
 1. Technology Heterogeneity
 2. Robustness
 3. Scaling
 4. Ease of Deployment
 5. Organizational Alignment
 6. Composability



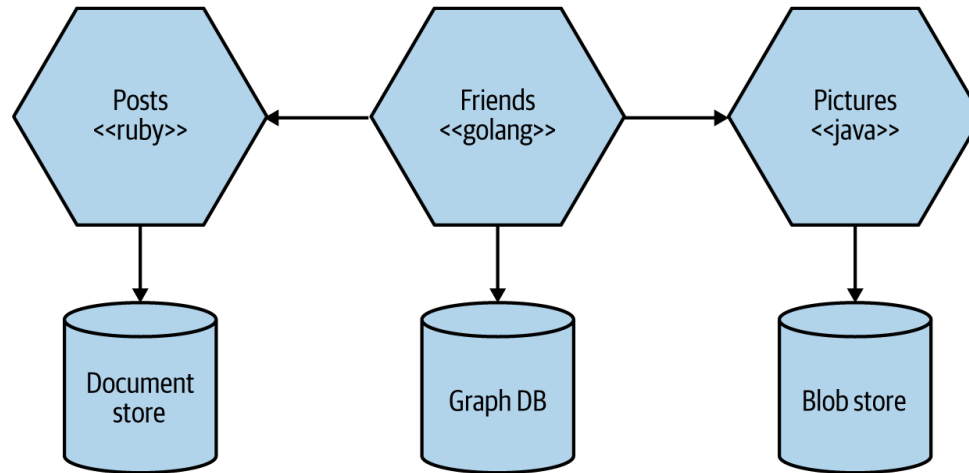
1) Technology Heterogeneity

- If one part of our system needs to improve its **performance**, we might decide to use a **different** technology stack that is better able to achieve the required performance levels.
- With a monolithic application, if I want to try a new programming language, database, or framework, any change will **affect** much of my system.
- With a system consisting of multiple services, I have multiple new places to try out a new piece of technology.



1) Technology Heterogeneity (cont.)

- Microservices can allow you to more easily embrace different technologies



1) Technology Heterogeneity (cont.)

- Embracing multiple technologies doesn't come without overhead, of course.
- Some organizations choose to place some **constraints** on language choices.
- Netflix and Twitter, for example, mostly use the Java Virtual Machine (JVM) as a platform because those companies have a very good understanding of the reliability and performance of that system.



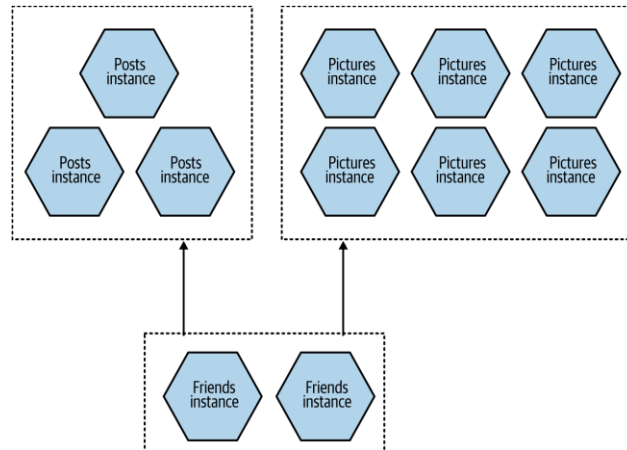
2) Robustness

- A key concept in improving the robustness of your application is the **bulkhead**.
- A component of a system may fail, but as long as that failure doesn't cascade, you can **isolate** the problem, and the rest of the system can carry on working.



3) Scaling

- With a large, monolithic service, we need to **scale everything** together.
- With smaller services, we can scale just those services that need scaling, allowing us to run other parts of the system on less powerful hardware.
- ***You can target scaling at just the microservices that need it***



4) Ease of Deployment

- A one-line change to a million-line monolithic application requires the **entire application** to be deployed in order to release the change.
 - That could be a large-impact, high-risk deployment.
- With microservices, we can make a change to a single service and deploy it independently of the rest of the system.
- This allows us to get our code deployed **more quickly**.



5) Organizational Alignment

- Many of us have experienced the problems associated with **large teams** and **large codebases**.
- Smaller teams working on smaller codebases tend to be more **productive**.
- Microservices also allow us to change ownership of services as the organization changes.



6) Composability

- One of the key promises of distributed systems is: *Opportunities for reuse of functionality*.
- With a monolithic application, if I want to break that up to get something more useful, I'll need a **hammer**!



Microservice Pain Points

- Developer Experience
 - You can't run the entire system on one machine.
- Technology Overload
 - You have to carefully balance the breadth and complexity of the technology you use against the costs that a diverse array of technology can bring.
- Cost
 - You will likely need to run more things—more processes, more computers, more network, more storage, and more supporting software.

Types of Headache

Migraine



Hypertension



Stress



Microservices



Microservice Pain Points (cont.)

- Reporting
 - With a monolithic system, you typically have a monolithic database.
 - With a microservice system, our data is scattered across multiple logically isolated schemas.
- Monitoring and Troubleshooting
 - With a monolithic system, if our CPU is stuck at 100% for a long time, we know it's a big problem.
 - With a microservice architecture with tens or hundreds of processes, can we say the same thing?
- Security
 - Much of our information flows over networks between services.
 - This can make our data more vulnerable to being observed.



Microservice Pain Points (cont.)

- Testing
 - With a microservice architecture, the scope of our end-to-end tests becomes *very* large.
 - We would now need to run tests across multiple processes.
- Latency
 - Information that previously flowed within only a single process now needs to be serialized, transmitted, and deserialized over networks.
- Data Consistency
 - Multiple processes manage state in different databases, causes potential challenges with respect to consistency of data.



Should I Use Microservices?

- Because of the numerous challenges I've outlined, adopting them still requires **careful thought**.
- You need to assess your own problem space, skills, and technology landscape and understand what you are trying to achieve before deciding whether microservices are right for you.
- Microservice architectures are often a bad choice for **brand-new products** or **startups**.
 - The domain that you are working with is typically undergoing **significant change**.



Should I Use Microservices? (cont.)

- For a small team, a microservice architecture can be difficult to justify because there is work required just to handle the deployment and management of the microservices themselves.
- After you understand where the **constraints** are in your architecture and what your pain points are—then you can focus your energy on using microservices in the most sensible places.



Should I Use Microservices? (cont.)

- The single biggest reason that organizations adopt microservices is to **allow for more developers** to work on the same system without getting in each other's way.
- Get your architecture and organizational boundaries right, and you allow more people to work independently of each other.
- Above all, a microservice architecture is one that can give you a lot of flexibility as you continue to evolve your system.



References

- Building Microservices: Designing Fine-Grained Systems,
Sam Newman
 - Chapter 1

